HireLoop Online Job Portal

Project report

Abstract

HireLoop is a web-based job portal that connects employers with job seekers under the supervision of an administrator. Employers post vacancies, review applications and message candidates. Job seekers search for jobs, upload a resume, apply and track the status of each application. Administrators manage users, approve or reject job listings, change system settings and watch statistics and activity. The system has three role-based dashboards, real user accounts, server-side access control, resume uploads and persistent storage. It runs on Node.js with no external dependencies and can be deployed to a container or a small server.

1. Introduction

Finding a job and hiring for one both involve repeated, manual work: posting openings, collecting resumes, tracking who has been contacted and who has been rejected. An online portal puts these steps in one place and gives each kind of user only the tools they need.

This project builds such a portal from a defined brief with three user types: Admin, Employer and Job Seeker. Each has its own dashboard and its own set of functions.

2. Objectives

- Let employers post, edit and delete job listings and manage the applications they receive.
- Let job seekers search and filter jobs, apply with a resume and cover letter, and track their applications.
- Let administrators control who can use the system, which jobs go live, and how the system is configured.
- Keep each role's data and actions separate and enforced on the server.
- Provide a deployable system, not only a prototype.

3. User Roles and Functions

Role	Main functions
Admin	Manage users and roles. Approve or reject job postings. Change system settings. View job statistics and a live activity log.
Employer	Post, edit and delete jobs. View posting history and statuses. Review applications and update their status. Message applicants. View application statistics.
Job Seeker	Search and filter jobs. Apply with a resume and cover letter. Track application status and history. Manage profile, skills and resume. See recommended jobs.

4. Requirements

4.1 Functional requirements

Admin

- Create, edit and delete users, and assign roles. A confirmation message is shown after each change.
- Approve or reject each employer job posting. The posting's status shows the decision.
- Update system settings: site name, whether new jobs need approval, whether registration is open, and the maximum resume size.
- View charts of jobs, applications and users, and a log of recent activity.

Employer

- Create a job with title, location, type, salary, description and requirements.
- Edit or delete their own jobs. An edited job returns to review when approval is required.
- Review each application, download the resume and set a status: Submitted, Reviewed, Shortlisted, Rejected or Hired.
- Send messages to applicants.

Job Seeker

- Register, sign in and keep a profile with name, skills and a resume (PDF or Word).
- Search by keyword, location and job type. Only approved jobs are visible.
- Apply once per job with a cover letter and receive a confirmation.
- See current applications, past results and employer messages.
- See recommended jobs based on the skills in the profile.

4.2 Non-functional requirements

- **Security:** passwords are never stored in plain text, and every action is checked against the user's role.
- **Reliability:** data survives a server restart.
- **Usability:** the interface works on desktop and mobile screens and supports light and dark colour schemes.
- **Portability:** the application needs only Node.js 18 or newer.

5. System Design

5.1 Architecture

The system is a single-page web client served by a Node.js HTTP server. The browser holds no authoritative data. After sign-in, the client asks the server for the data that the signed-in user is allowed to see, draws the dashboard for that role, and sends every change back to the server as an action.

1. The browser loads one HTML page.
2. The user signs in or registers. The server sets a signed, HttpOnly session cookie.
3. The client requests its state from the server. The server returns only the records that role may see.
4. Each user action is sent as a request. The server checks the role, validates the input, changes the data and returns a confirmation message.
5. The client reloads its state and redraws.

5.2 Data model

- **Users:** id, name, email, role, password hash, and for employers a company name; for job seekers, skills and the resume file name.

- **Jobs:** id, owner, company, title, location, type, salary, description, requirements and status (pending, approved or rejected).
- **Applications:** id, job, applicant, status, cover letter and the resume name. A copy of the resume file is kept with each application so it does not change if the applicant later uploads a new one.
- **Messages:** the application they belong to and the text.
- **Activity log and settings.**

5.3 What each role can see

- An admin receives all users, jobs and applications.
- An employer receives only their own jobs, the applications to those jobs, and the applicants' details.
- A job seeker receives approved jobs, their own applications, and their own profile.

5.4 Interface

The server exposes a small set of endpoints: sign-in, registration, sign-out, state retrieval, a single action endpoint for all changes, resume download and a health check. Each action name is mapped to the role or roles allowed to use it.

5.5 Job lifecycle

An employer posts a job. If approval is required, it is pending until an admin approves or rejects it. Only approved jobs appear to job seekers. A job seeker applies, and the application moves through Submitted, Reviewed and Shortlisted to Hired or Rejected, set by the employer. The employer can message the applicant at any stage, and the applicant sees the status and messages on their dashboard.

6. Implementation

Front end. A single HTML file with plain JavaScript and CSS. It renders a role-specific navigation and a screen for each function. User-supplied text is escaped before it is drawn, which prevents script injection.

Back end. A Node.js server using only built-in modules. It handles routing, sessions, validation, file storage and role checks.

Storage. Records are kept in a JSON file that is written to disk after each change, using a temporary file and an atomic rename so a crash cannot leave a half-written file. Resume files are stored in a separate folder.

Recommendations. Each approved job is scored by how many of the job seeker's skills appear in its title, requirements or description. Jobs with at least one match are listed, best match first.

Statistics. Bar charts show jobs by status, applications by status, users by role and applications per job. Employers see applicants per job and their application pipeline.

7. Security

- **Passwords** are hashed with scrypt and a random salt per user.
- **Sessions** use a signed cookie that cannot be read by page scripts, expires after seven days and is marked Secure when the site is served over HTTPS.
- **Access control** is enforced on the server for every action. A job seeker who sends an admin request is refused.
- **Brute-force protection:** sign-in attempts are limited per address.
- **Cross-site request protection:** state-changing requests must be JSON, and the session cookie is SameSite.
- **Uploads:** only PDF and Word files are accepted, the file's first bytes are checked against the type, and the size is limited by an admin setting.
- **Response headers** restrict where scripts and styles may load from and block framing and content sniffing.
- **Input limits** cap the length of every text field and the size of each request.

8. Testing

The system was tested by driving the real server with an automated browser and with direct requests. The scenarios covered:

- Registering an employer and a job seeker, and signing in as the admin; a wrong password is refused.
- Posting a job and confirming that it is hidden from job seekers until the admin approves it.
- Searching for jobs, refusing an application without a resume, rejecting a fake PDF, and accepting a real one.

- Applying, downloading the resume as the employer, changing the application status and sending a message that the job seeker then sees.
- Staying signed in after a page reload, and signing out.
- Confirming that data and resumes remain after the server is restarted.
- Confirming that a job seeker cannot approve jobs, delete users or change settings, and that requests with no session or a forged session are refused.

These were end-to-end checks run during development. A permanent automated test suite is not included and would be a useful addition.

9. Deployment

The project includes a Dockerfile, a Render blueprint and a README. Configuration is through environment variables: the port, the data folder, a secret for signing sessions, and the first admin's email and password. Data must be stored on a persistent disk, and the site should be served over HTTPS. The health check endpoint lets hosting platforms monitor the service.

10. Limitations and Future Work

- Storage in a single JSON file suits one server and a modest number of users. A database such as PostgreSQL or MongoDB would allow many concurrent users and several servers.
- There is no email. Verification, password reset and notifications of status changes are natural next steps.
- Recommendations use simple skill matching and could use ranking or machine learning.
- Further additions: saved jobs and alerts, company profiles, resume parsing, pagination of long lists and an automated test suite.

11. Conclusion

HireLoop delivers every function in the brief for all three roles, with real accounts, server-enforced permissions, resume handling and persistent data, packaged so it can be deployed with a few commands. It forms a working base that can grow in capacity and features as the need arises.

Appendix A: User Guide

Everyone opens the same web address and signs in. The app then shows a different dashboard depending on the person's role.

A.1 Owner set-up

1. Run the server and open the address (http://localhost:3000 on a local computer, or the website address once deployed).
2. Sign in as **Admin** with the email and password set at startup.
3. Open **Settings** and choose whether new jobs need approval and whether registration is open.

A.2 Employer

1. Click **Create an account**, choose **I am hiring**, and enter a company name.
2. Open **Post a job** and fill in the title, location, type, salary, description and requirements.
3. The job waits for the admin's approval, then goes live.
4. Under **Applications**, open each applicant, download the resume and set a status: Reviewed, Shortlisted, Rejected or Hired.
5. Send a message to a candidate from the same screen.
6. Use **My jobs** to edit or delete listings and **Statistics** to see how many people applied.

A.3 Job seeker

1. Click **Create an account** and choose **I am looking for a job**.
2. In **Profile**, add skills and upload a resume (PDF or Word). A resume is required before applying.
3. In **Find jobs**, search by keyword, location or job type, then click **Apply** and write a cover letter.
4. In **My applications**, follow the status of each application and read messages from employers. Finished applications move to History.
5. **Recommended** lists jobs that match the skills in the profile.

A.4 Admin

- **Job listings:** approve or reject each job. Job seekers see only approved jobs.
- **Users:** add, edit or delete accounts and change roles.

- **Statistics** and **Live activity** show how the site is used.
- **Settings:** change the site name, the approval rule, registration and the resume size limit.

Anyone who registers can only become an employer or job seeker, never an admin. There is no email yet, so users must sign in to see new statuses and messages, and a forgotten password is reset by the admin from the Users tab.

A.5 Demonstration plan

Open three browser windows (a private window or a different browser for each, because one browser keeps only one login). Sign in as the admin in the first, register as an employer in the second and as a job seeker in the third. Then follow this order: the employer posts a job, the admin approves it, the job seeker uploads a resume and applies, and the employer updates the status and sends a message that the job seeker then sees.

Appendix B: Deployment Guide

To put the portal online, use an always-on server with a **persistent disk** and **HTTPS**. This needs the owner's own hosting account.

B.1 Option 1: Render

1. Upload the project folder to a new GitHub repository.
2. At render.com choose **New, then Blueprint**, and connect the repository. Render reads the included `render.yaml`.
3. Enter `ADMIN_EMAIL` and `ADMIN_PASSWORD` when asked. `SESSION_SECRET` is generated automatically.
4. Click **Apply**. Render builds the service and gives an address ending in `onrender.com`.
5. Open the address and sign in as admin. The address followed by `/healthz` should reply with `{"ok":true}`.

The disk keeps data between restarts, and Render requires a paid plan for it, so check current pricing. On a plan with no disk, all users, jobs and resumes are lost each time the app restarts.

B.2 Option 2: Own server with Docker

This needs a Linux server with Docker and a domain name pointing to it.

```
docker build -t hireloop .
docker run -d --name hireloop --restart unless-stopped -p 3000:3000 \
  -v hireloop-data:/data \
  -e SESSION_SECRET=put-a-long-random-string-here \
  -e ADMIN_EMAIL=you@example.com \
  -e ADMIN_PASSWORD=choose-a-strong-password \
  hireloop
```

Add HTTPS with Caddy, which obtains the certificate automatically:

```
caddy reverse-proxy --from yourdomain.com --to localhost:3000
```

Railway and Fly.io also work: deploy the Dockerfile and attach a volume mounted at `/data`.

B.3 After going live

- **Back up** the data folder (`/data` on the server) regularly.
- **Keep the admin password private.** It can be changed later in the Users tab.
- **Use HTTPS only.** Login cookies are marked secure automatically over HTTPS.
- **First start:** if no admin password was set, a random one is printed in the server logs on the line starting with "First run".